

OPENMP application

Réalisée par Firdawsse Ahchouche

Encadré par: Mr BENBA

Code 1 :

```
#include <omp.h>
#include <stdio.h>
int main()
{
    #pragma omp parallel
    {
        printf ("Bonjour!\n ");
    }
    return 0;
}
```

```
firdaws@Firdaws:~$ ./tp
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
firdaws@Firdaws:~$ |
```

Commentaire :

Ce code utilise la bibliothèque OpenMP pour créer une région parallèle où plusieurs threads peuvent exécuter du code en parallèle.

OpenMP simplifie la parallélisation par rapport à l'utilisation directe des threads. Lorsque vous créez une région parallèle avec OpenMP, le code à l'intérieur est exécuté simultanément par plusieurs threads. Ainsi, si vous imprimez le message "Bonjour !" à l'intérieur de cette région parallèle, il sera affiché plusieurs fois, chaque thread exécutant la fonction printf de manière simultanée. Le nombre de sorties "Bonjour !" dépend alors du nombre de threads créés par la région parallèle.

Code 2 :

```
#include <omp.h>
#include <stdio.h>
int main()
{
#pragma omp parallel num_threads(20)
{
printf ("Bonjour!\n ");
}
return 0;
}
```

```
firdaws@Firdaws:~$ ./tp
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
Bonjour!
firdaws@Firdaws:~$ |
```

Commentaire :

Ce programme utilise la bibliothèque OpenMP pour créer une zone où plusieurs tâches peuvent être exécutées simultanément, chacune étant gérée par un thread distinct. Plus précisément, il crée une région parallèle avec 20 threads.

Code 3 :

```
#include <omp.h>
#include <stdio.h>

int main() {
    #pragma omp parallel num_threads(20)
    {
        printf("Bonjour! Je suis le thread %d\n", omp_get_thread_num());
    }
    return 0;
}
```

Commentaire :

Ce code crée une région parallèle en utilisant OpenMP, où chaque thread parallèle imprime un message indiquant son numéro de thread. À l'intérieur de cette région parallèle, la directive `num\_threads(20)` spécifie explicitement l'utilisation de 20 threads pour exécuter le code parallèle. Chaque thread parallèle obtient ensuite son numéro de thread en utilisant la fonction `omp\_get\_thread\_num()` et imprime ce numéro ainsi qu'un message "Bonjour! Je suis le thread".

```
firdaws@Firdaws:~$ ./tp
Bonjour! Je suis le thread 1
Bonjour! Je suis le thread 9
Bonjour! Je suis le thread 11
Bonjour! Je suis le thread 12
Bonjour! Je suis le thread 13
Bonjour! Je suis le thread 3
Bonjour! Je suis le thread 14
Bonjour! Je suis le thread 4
Bonjour! Je suis le thread 15
Bonjour! Je suis le thread 17
Bonjour! Je suis le thread 0
Bonjour! Je suis le thread 2
Bonjour! Je suis le thread 7
Bonjour! Je suis le thread 16
Bonjour! Je suis le thread 18
Bonjour! Je suis le thread 19
Bonjour! Je suis le thread 6
Bonjour! Je suis le thread 8
Bonjour! Je suis le thread 10
Bonjour! Je suis le thread 5
firdaws@Firdaws:~$
```

Commentaire :

On remarque que lors de l'exécution, chaque thread imprime son numéro de manière indépendante. Cependant, l'ordre dans lequel les threads affichent leur numéro peut varier, car ils s'exécutent en parallèle. Cette variation résulte du fonctionnement du système et de la façon dont l'ordonnanceur de tâches gère les threads.

Code 4 :

```
#include <stdlib.h>
#include <omp.h>
#include <stdio.h>

#define THREADS 8
#define N 10

int main() {
    int i;
    #pragma omp parallel for num_threads(THREADS)
    for (i = 0; i < N; i++) {
        printf("Thread %d is doing iteration %d. \n", omp_get_thread_num(), i);
    }
    /* all threads done */
    printf("All done!\n");
    return 0;
}
```

Commentaire :

Ce code utilise OpenMP pour exécuter une boucle for en parallèle sur plusieurs threads. Il commence par initialiser les bibliothèques nécessaires, puis définit le nombre de threads à utiliser ainsi que le nombre d'itérations dans la boucle. Ensuite, il divise les itérations de la boucle entre les threads. Chaque thread exécute une partie de la boucle for, imprimant le numéro du thread et l'itération qu'il traite. Une fois que tous les threads ont terminé leur exécution, le message "All done!" est affiché.

```
firdaws@Firdaws:~$ ./tp
Thread 0 is doing iteration 0.
Thread 6 is doing iteration 8.
Thread 0 is doing iteration 1.
Thread 7 is doing iteration 9.
Thread 5 is doing iteration 7.
Thread 1 is doing iteration 2.
Thread 1 is doing iteration 3.
Thread 3 is doing iteration 5.
Thread 2 is doing iteration 4.
Thread 4 is doing iteration 6.
All done!
```

Commentaire :

La boucle for permet de déterminer équitablement les itérations sous les différents thread (cpu , cpu, cpu..) lancé.

Si On élimine le for :

```
#include <omp.h>
#include <stdio.h>
int main (int argc, char *argv[])
{
#pragma omp parallel
{
printf ("Bonjour! Je suis le thread %d sur %d threads.\n", omp_get_thread_num(), omp_get_num_threads ());
}
return 0;
}
```

Commentaire :

Ce code utilise OpenMP pour créer une région parallèle où chaque thread imprime un message indiquant son numéro de thread ainsi que le nombre total de threads en cours d'exécution.

Pour cela :

- Une région parallèle est créée à l'aide de la directive `#pragma omp parallel`, ce qui indique au compilateur que le bloc de code suivant doit être exécuté en parallèle.
- À l'intérieur de cette région parallèle, chaque thread exécute la tâche spécifiée, qui consiste à imprimer un message contenant son numéro de thread (obtenu via `omp\_get\_thread\_num()`) ainsi que le nombre total de threads (obtenu via `omp\_get\_num\_threads()`).

```
firdaws@Firdaws:~$ gcc tp.c -fopenmp -o tp
firdaws@Firdaws:~$ ./tp
Bonjour! Je suis le thread 12 sur 16 threads.
Bonjour! Je suis le thread 0 sur 16 threads.
Bonjour! Je suis le thread 15 sur 16 threads.
Bonjour! Je suis le thread 2 sur 16 threads.
Bonjour! Je suis le thread 7 sur 16 threads.
Bonjour! Je suis le thread 14 sur 16 threads.
Bonjour! Je suis le thread 8 sur 16 threads.
Bonjour! Je suis le thread 13 sur 16 threads.
Bonjour! Je suis le thread 6 sur 16 threads.
Bonjour! Je suis le thread 4 sur 16 threads.
Bonjour! Je suis le thread 11 sur 16 threads.
Bonjour! Je suis le thread 5 sur 16 threads.
Bonjour! Je suis le thread 1 sur 16 threads.
Bonjour! Je suis le thread 10 sur 16 threads.
Bonjour! Je suis le thread 9 sur 16 threads.
Bonjour! Je suis le thread 3 sur 16 threads.
firdaws@Firdaws:~$ |
```

Code 5 :

```
#include <omp.h>
#include <stdio.h>
int main ()
{
    int x = 0;
    #pragma omp parallel
    {
        x= x + 1;
        int y = x + 1;
        printf("x=%d y=%d\n", x, y);
    }
}
```

Commentaire :

Ce code crée une région parallèle où chaque thread modifie une variable x et calcule une autre variable y en utilisant la valeur modifiée de x. Il peut y avoir des problèmes de concurrence car plusieurs threads peuvent accéder à x simultanément. Ensuite, les valeurs de x et y sont imprimées pour chaque thread.

```
firdaws@Firdaws:~$ ./tp
```

```
x=5 y=6
x=5 y=6
x=1 y=2
x=5 y=6
x=6 y=7
x=2 y=3
x=4 y=5
x=9 y=10
x=2 y=3
x=6 y=7
x=3 y=4
x=5 y=6
x=5 y=6
x=7 y=8
x=10 y=11
x=8 y=9
```

Dans ce code, la variable x est partagée entre les threads. Les valeurs de y dépendent de $x + 1$, mais les opérations sur x peuvent être affectées par l'ordonnancement des threads et les conditions de course.

L'output affiche les valeurs de x et y pour chaque thread. Comme les opérations sont exécutées en parallèle par plusieurs threads, les valeurs de x peuvent être mises à jour par différents threads à différents moments, conduisant à des résultats inattendus et variant à chaque exécution du programme.

Code 6 :

```
#include <omp.h>
#include <stdio.h>
int main ()
{
    int x = 0;
    #pragma omp parallel private(x)
    {
        x= x + 1;
        int y = x + 1;
        printf("x=%d y=%d\n",x,y);
    }
}
```

Commentaire :

Ce code utilise OpenMP pour créer une région parallèle. Chaque thread parallèle incrémente sa propre copie locale de la variable x de 1, puis calcule y en utilisant cette copie locale de x. Enfin, chaque thread imprime les valeurs de x et y qu'il a calculées.

```
firdaws@Firdaws:~$ ./tp
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1188155457 y=1188155458
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
x=1 y=2
firdaws@Firdaws:~$
```

Code :

```
#include <stdio.h>
#include <omp.h>

int main() {
    int N = 100;
    int sum = 0;

    #pragma omp parallel for reduction(+:sum)
    for(int i = 1; i <= N; i++) {
        sum += i;
    }

    printf("La somme des entiers de 1 à %d est : %d\n", N, sum);

    return 0;
}
```

Dans ce code on a une réduction, on prend le résultat de chaque thread et on les réduit à une somme.

Reduction(+:sum) : on a une réduction de l'opération, c'est la sum qui va prendre cette valeur.

```
firdaws@Firdaws:~$ gcc tp.c -fopenmp -o tp
firdaws@Firdaws:~$ ./tp
La somme des entiers de 1 à 100 est : 5050
firdaws@Firdaws:~$
```

Programmation Parallèle :

Code 8 :

```
#include <stdio.h>
#define N 10000
float PI, x;
int i;

int main()
{
    for(i=0; i<N; i++){
        x=(float)i/N;
        PI+=(4/(1+x*x))/N;
    }
    printf("PI=%.16f\n", PI);
    return 0;
}
```

Commentaire :

Ce code utilise la méthode de l'intégration numérique pour estimer la valeur de Pi de manière séquentielle.

```
firdaws@Firdaws:~$ ./tp
PI=3.1416923999786377
firdaws@Firdaws:~$
```

Code 9 :

```
GNU nano 6.2
#include <stdio.h>
#include <time.h>

#define N 10000

int main() {
    float PI = 0.0f;
    float x;
    clock_t t1, t2;
    double clk = CLOCKS_PER_SEC;

    t1 = clock();

    for (int i = 0; i < N; i++) {
        x = (float)i / N;
        PI += (4 / (1 + x * x)) / N;
    }

    t2 = clock();

    printf("PI=%.16f\n", PI);

    double temps = (t2 - t1) / clk;
    printf("Le temps est %f secondes\n", temps);

    return 0;
}
```

Commentaire :

Ce code mesure également la durée nécessaire pour effectuer le calcul. Il calcule cette durée en soustrayant le temps t1 du temps t2, puis convertit le résultat en secondes en le divisant par CLOCKS_PER_SEC, et enfin affiche le résultat.

```
firdaws@firdaws:~$ ./tp
PI=3.1416923999786377
Le temps est 0.000020 secondes
firdaws@Firdaws:~$ |
```

Le résultat affiché montre une estimation de Pi d'environ 3.1416923999786377 et indique que le calcul a pris environ 0.000020 secondes.

Code 10 :

```
#include<stdio.h>
#include <time.h>

#define BILLION 1000000000L
#define N 1000000000

float PI;
double x;
int i;
int main(){
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (i=0;i<N;i++){
        x=(float)i/N;
        PI+=(4/(1+x*x))/N;
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_spent = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / (double)BILLION;
    printf("Valeur de pi approximative : %.16lf\n", PI);
    printf("Temps d'execution : %.9lf secondes\n", time_spent);

    return 0;
}
```

Commentaire :

Ce code mesure le temps de calcul en utilisant la fonction `clock_gettime()`. Il commence par utiliser `clock_gettime(CLOCK_MONOTONIC, &start)` pour obtenir le temps au début du calcul, puis `clock_gettime(CLOCK_MONOTONIC, &end)` pour obtenir le temps à la fin du calcul. Ensuite, il calcule la différence entre ces deux temps pour obtenir le temps écoulé, qu'il convertit ensuite en secondes.

```
firdaws@Firdaws:~$ ./tp
Valeur de pi approximative : 0.125000000000000000
Temps d'execution : 4.211705176 secondes
```

Le résultat affiché donne une estimation de Pi d'environ 0.125000000000 indique que le calcul a pris environ 4.211705176secondes.